```
        charge_decline_reason: payments_response.explanation
      )
    end
  end
  order
end
```

The methods `charge`, `send_email`, and `request_fulfillment` wrap the calls to those three services (you may examine the entire file if you want to see how that works). We can think about the process as having two parts. The first validates the order, and the user has to wait for that to complete. The second part is to handle payments, email, and order fulfillment and the user does *not* have to wait for those to complete. Let's change the code so that this second part of the logic is triggered from a job called `CompleteOrderJob`.

You don't want a lot of logic in the job classes. They should act like controllers in that they make sense of the arguments given, then defer execution to another class. In our case, `OrderCreator` exists already, so assume a method named `complete_order` will exist and write your job like so:

snapshots/1-2/sidekiq-book/app/jobs/complete_order_job.rb

```
class CompleteOrderJob
  include Sidekiq::Job

  def perform(order_id)
    order = Order.find(order_id)
    OrderCreator.new.complete_order(order)
  end
end
```

`complete_order` will basically do everything from the original `create_order` method after the model passes validation.

snapshots/1-2/sidekiq-book/app/services/order_creator.rb

```
    end

»   def complete_order(order)
»     payments_response = charge(order)
```